Functions

ID1063 Introduction to Programming in C

Instructor: Rakesh Venkat

Week 7 • Lectures 13,14

Week 7 at a glance

Lecture 13: functions and control flow
Lecture 14: functions with arrays and strings

Written midsem: 16 Sep
Lab Exam 1: 19 Sep
No Week 8 lab

Please fill out the form shared by Prof. Aravind yesterday regarding how you are finding the course until now.

A simple problem

- Print a clean log reportwith a row of stars between each line

```
*****************************
   ATMOSPHERIC DATA LOG
*****************************
Status: All sensors operational.
*****************************
   END OF REPORT
*****************************
```

- How would you print every row of *’s?

Functions

- Functions are named, reusable blocks of code that accomplish a certain task

Instead of one long main:

- name a task, write function for it
- test it in isolation
- reuse it when needed

Familiar functions

```
printf("Hello\n");
scanf("%d", &age);
```

- Code for these functions have already been written in the stdio library

main() : is a special function, and used by the Operating System when executing the program

How to write a function

```
return type    name      parameters/arguments
    ↓          ↓           ↓
int square(int value) {
    return value * value;
}
```

- return type: what the function gives as a result
- name: name of the function
- parameters or arguments: What the function acts on

- Body: the statements that do the job.

Example 1: First use of functions

```
#include <stdio.h>

void showWelcome(void) {
    printf("Welcome to ID1063!\n");
}

int main(void) {
    showWelcome();
    return 0;
}
```

void : means no value is used/returned

Function execution

```c
#include <stdio.h>

void showWelcome(void) {
    printf("Welcome to ID1063!\n");
}

int main(void) {
    showWelcome();
    return 0;
}
```

1. main reaches call to showWelcome()
2. control moves into showWelcome
3. showWelcome function body is executed
4. control returns to main, proceeds execution from next line

Why use functions

- **Avoid repetition:** Functions allow you to write code once and use it many times.
- **Readability:** main() now reads like plain English (printBorder()), making the core program clear
- **Maintainability:** To change the border style or length, you only modify the code inside printBorder() once.

Example 2: Functions with return values

```
int square(int value) {
    return value * value;
}

int main(void) {
//take input from user
//call the function here to compute n^2
}
```

Functions may compute a value that the main program can use

These are stored in a predetermined location that main knows about.

Function execution

1. main reaches call to square(…)
2. control moves into square
3. square is computed and is stored in a predetermined location known to main (return statement)
4. control returns to main, proceeds execution from next line

return is not printf

return value * value;
- sends a result back to the caller

printf("%d\n", value);
- displays characters on screen

A printed result is not automatically stored.

Definition vs call

Definition: describes the job once

```
int square(int value) {
    return value * value;
}
```

Call: uses the job, as we have done in the main program

```
int answer = square(7);
```

Example 3: Finding the maximum

```
int maximum(int first, int second) {
    if (first > second) {
        return first;
    }
    return second;
}
//in main:
int largest = maximum(73, 68);
```

As soon as the program control encounters a return statement, it returns back to the main

For e.g. when it finds that 73>68, it executes the statement ‘return first’, and returns 73 and comes back to the main program. This is why an ‘else’ is not needed in the function when we return second.

Parameters vs arguments

Definition:
int maximum(int first, int second)

Call:
maximum(scoreA, scoreB)

parameter = variable in the definition

argument = value or expression used in the call

Lec 14: Functions II

Lab 5,6 problem discussion

Code written for these can be found in the Week 7 codes folder on google drive.

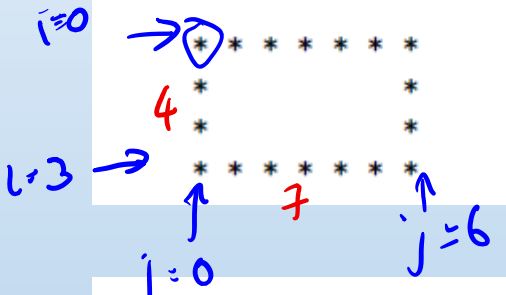
Lab 5: P4, P6

4. Write a program to accept two integers m and n and display the perimeter of a $m \times n$ rectangle.

   Example run:

   Enter the value of m: 4 → row

   Enter the value of n: 7 → col



6. Write a program that accepts a positive integer n, followed by n integers stored in an array. Determine whether there are two distinct positions in the array containing the same value.

   If there are two such positions, print one such pair and the corresponding common value. Otherwise, print that the array has no repeated values.

   For example:

```
Enter the value of n: 6
Enter the values: 4 7 2 9 7 5
Repeated value: 7, Positions: 1, 4
```

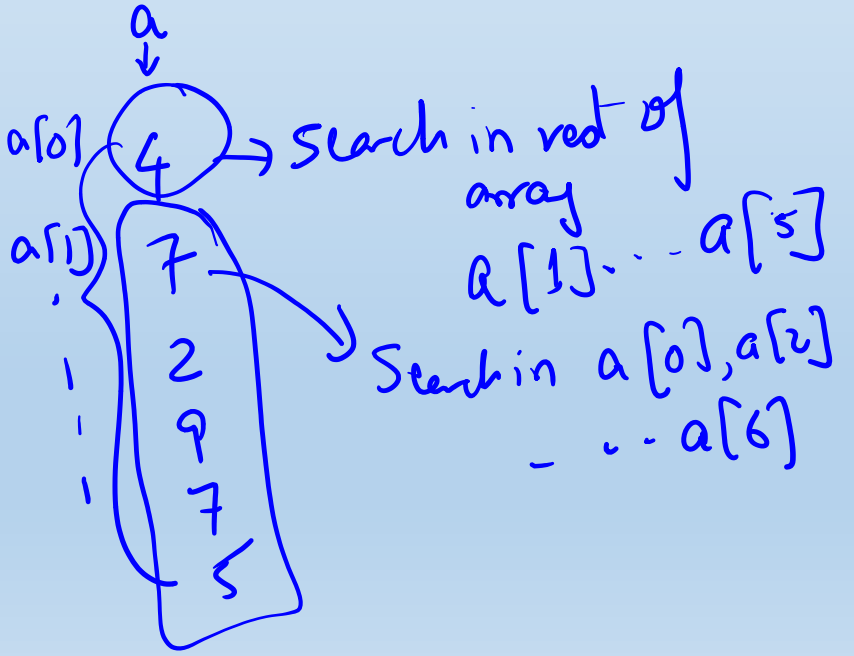
① Nested loops (i, j) → count

② i = 0 || (logical OR)

✓ i = 0 ||

j = 6 ||

i = 3 ||



Some general tips

- First check if you can relate the given problem to something you have already seen
- Think of how / where you need to modify to get the desired result.
- Break the problem into clear logical subparts.
- Write code for the subparts, testing the code at partial steps instead of the very end.
- Test on inputs, especially boundary cases (e.g. all elements equal, negative elements, boundary cases etc. depending on the question)

Lab 6: P4 (first occurrence)

4. Write a C program to find the index of the first occurrence of a given character in a string. An example is shown below.

```
Input: banana
character: n
Output: 2
```

If the character is not present, output -1.

Some points:

Make sure to keep a space before %c or %s when getting a string/character input (to account for newline/space is prior input)

Lab 6: P5 (palindrome)

5. Write a C program to check whether a given string is a palindrome (reads the same forwards and backwards). Assume that all characters are English and lowercase.

```
Input:madam
Output: Palindrome

Input: hello
Output: Not a Palindrome
```

Solution idea: Split the steps into logical parts:

1. Find length of string
2. Check palindrome condition

Some points to be careful about:

1. Decide boundary conditions to implement the correct check. Here there are two places: the termination criterion of the loop (why i<len/2 works? Will writing i<len work?)

2. Why we check str[i] != str[len-i-1], and not str[i] != str[len-i]
3. Use of a flag/indicator variable that is set to 1 or 0 to record the result of the loop iterations.

Recap

```
int maximum(int first, int second) { //function definition
    if (first > second) {
        return first;
    }
    return second;
}

int main() {
    int a = 73, b;
    scanf(“%d”, &b);
    int largest = maximum(73, b);  // function call
    printf(“Largest number is %d”, largest);
}
```

Local variables

```
int square(int value) {
    int ans = value * value;
    return ans;
}
```

Q: Is the variable ‘ans’ available in the main program? If we did printf(“%d”, ans) in the main program, what would happen?

Pass by value

```
void addOne(int number) {
    number = number + 1;
}

int main(void) {
    int count = 4;
    addOne(count);
    printf("Count value is: %d \n",count);
    return 0;
}
```

Q: What is printed by the program?

A: Normal variables are passed by *value*: i.e. **only a copy is passed to the function**
So, the printed output is still 4, since addOne works on the copy of the count variable.